

EECE 2140

Computing Fundamentals for Engineers

System Design Overview

Battleship

A C++ Engineering Project

 **Team Members:** Elliott Hill

Aiden Powell

 **Instructor:** Dr. Fatema Nafa

Semester: Spring 2026

 **Repository:** <https://github.com/AidenPowell14/eece2140-battleship>

Contents

1	Project Overview	2
1.1	Motivation	2
1.2	Project Goals	2
2	Basic Functionalities	2
2.1	Functionality Summary Table	2
2.2	Detailed Functionality Descriptions	3
3	Object-Oriented Programming (OOP) Design	4
3.1	OOP Design Table	4
3.2	Key OOP Requirements	7
4	System Overview — Class Relationships	7
4.1	High-Level Description	7
4.2	UML Class Diagram	8
4.3	Detailed Description	8
5	Pseudo Code	9
6	C++ Implementation Highlights	10
7	Challenges, Solutions & Lessons Learned	11
7.1	Challenges & Solutions	11
7.2	Lessons Learned	11
8	Limitations & Future Work	12
9	GitHub Repository Structure	13
A	Sample Program Output	13
B	Data Maintenance	15

1 Project Overview

💡 Project Description

Battleship is a C++ program that recreates the popular two player game of the same name. It keeps track of where each player has placed their ships, where a player has attacked, and stores whether said attack is a hit, a miss, and if it sinks a ship or not. The project demonstrates core object-oriented programming principles and the appropriate use of STL data structures (`vector`, `unique_ptr`) to replicate a real-world game.

1.1 Motivation

While perhaps not a highly-application engineering simulation, this project serves as an exercise in management of complex object-oriented relationships and data integrity over the course of a dynamic and user-driven program. Elements of game design and UX were also explored; the project revealed challenges in user interfaces for two players on one display.

1.2 Project Goals

- ✓ Imitate a board game and its mechanics
- ✓ Apply OOP principles: encapsulation, inheritance, and polymorphism.
- ✓ Use STL data structures (`vector`, `enum`) appropriately and efficiently.
- ✓ Collaborate via GitHub using feature branches and pull requests.

2 Basic Functionalities

This section lists all core functionalities of the game together with their expected inputs, outputs, and contribution to the overall system.

2.1 Functionality Summary Table

#	Functionality	Description	Input	Output
F1	Start Game	Allows user to choose grid size, number of ships, types of ships	Number of and sizes of ships from a file, board dimensions	Prompt to begin ship placement
F2	Ship Placement	Allows user to place ships on the game board	Starting position and orientation (grows right or down)	Terminal confirmation of success
F3	Strike	Lets user attack a grid point	Position to strike	Hit or miss marker on grid

2.2 Detailed Functionality Descriptions

F1 — Start Game

Description: Program starts from the command line with arguments regarding the board size and the ships to play with for each player. After initializing, the program will display a screen discussing input syntax and basic rules. Each player can start with a different set of linear ships. Ship files must contain a series of whitespace-separated integers representing the sizes of each of their ships.

Input: number of rows, number of columns, filepath to ships for each player

Output: Begins first prompt for ship placement from each player

Contribution: Launches the program

F2 — Ship Placement

Description: In order, players are prompted to add their ships to their own boards. After this phase, it is impossible to edit one's ship configuration. As explained in the rules, ships grow either downward or rightward (depending on orientation). Position requests that are out of bounds or would intersect with preexisting ships are rejected. Once one player has placed all of their ships, a mutual confirmation screen prompts player transition acknowledgment, and then the process is repeated for the other player.

Input: Row, column, and orientation

Output: Confirmation of successful ship placement and proceeding to the next

Contribution: Sets the game state for the rest of the game.

F3 — Strike

Description: Players take turns firing missiles at locations on each other's boards until one player is left without any ships left. Each guess immediately returns a hit/miss report to the attacker, and then prompts another mutual confirmation screen. If a complete battleship would be sunk as the result of an attack, an additional notification is raised, and the end of game condition is checked. An opponent's strikes against a player are not displayed on the victim's HitBoard unless the strike resulted in a direct hit.

Input: Row and column

Output: Hit/miss report, sunken battleship notification, end of game notification

Contribution: Primary gameplay loop revolves around this process

3 Object-Oriented Programming (OOP) Design

3.1 OOP Design Table

Class	Attribute / Method	Access	Data Type	Description
Controller	preface()	public	void	Tells the user rules
	switchPlayer()	public	void	Switched between active player and inactive player
	Controller(Model&)	public		constructor
	startAttacks	public	void	Initiates attack gameplay loop until game end
	displayError	public	void	Prints error when an input is invalid
	promptPlaceShip	public	void	Prompts the active player to place their ship on the board, and makes sure it is a legal placement
	sunkBattleship	public	void	Checks if game is over and announces when a ship has been sunk
	activePlayer	private	Color	Maintains player turn order

Class	Attribute / Method	Access	Data Type	Description
	model	private	Model&	Model object to manipulate game state
	gameOver	private	bool	Check if game ending protocol should start
	winner	private	Color	Current winner or "null"
HitBoard	owner	private	Color	player owner of board
	numRows	private	int	contains the number of rows on the board
	numCols	private	int	contains the number of columns on the board
	HitBoard(Color, int, int)	public		constructor
	get()	public	vector<vector<HitStatus>>&	Retrieves grid of statuses
	get(Point)	public	HitStatus	Retrieves status at point
	struck(Point, HitStatus)	public	void	sets status at given point
Model	redProfile	private	Profile	Red player info
	blueProfile	private	Profile	Blue player info
	activeProfile	private	Profile*	Active player info
	inactiveProfile	private	Profile*	Inactive player info
	numRows	private	int	Board rows
	numCols	private	int	Board cols
	controller	private	Controller*	Listening controller
	started	private	bool	Game started flag
	Model()	public		constructor
	start(int, int, vector<int>, vector<int>)	public	void	Starts ship placement with given ships and board size
	strike(Point)	public	HitStatus	Fires a missile at position

Class	Attribute / Method	Access	Data Type	Description
	setShip(Point, int, bool)	public	void	Places ship starting at given position
	isGameOver()	public	Color	Returns current winner or NEITHER
	getHits(Color)	public	vector<vector<HitStatus>>&	Retrieves grid of statuses for player
	getShips(Color)	public	vector<unique_ptr<Ship>>&	Retrieves list of ships for player
	rows()	public	int	Gets board row count
	cols()	public	int	Get board column count
	setListener(Controll	public	void	Subscribes controller to this model
ShipBoard	player	private	color	stores the color assigned to a player
	sealed	private	bool	when false, lets the ships on the board be moved. once true, ships on board can no longer change position
	ships	private	vector<unique_ptr<Ship>>	all ships for this player
	numRows	private	int	row count of board
	numCols	private	int	column count of board
	ShipBoard	public		constructor
	seeShips()	public	vector<unique_ptr<Ship>>&	Retrieve list of all ships for player
	addShip(Point, int, bool)	public		places ship at desired starting point
	seal()	public	void	Prevents further modification to the board

Class	Attribute / Method	Access	Data Type	Description
ShipReader	read(string)	public	vector<int>	Reads config files for ships for play

3.2 Key OOP Requirements

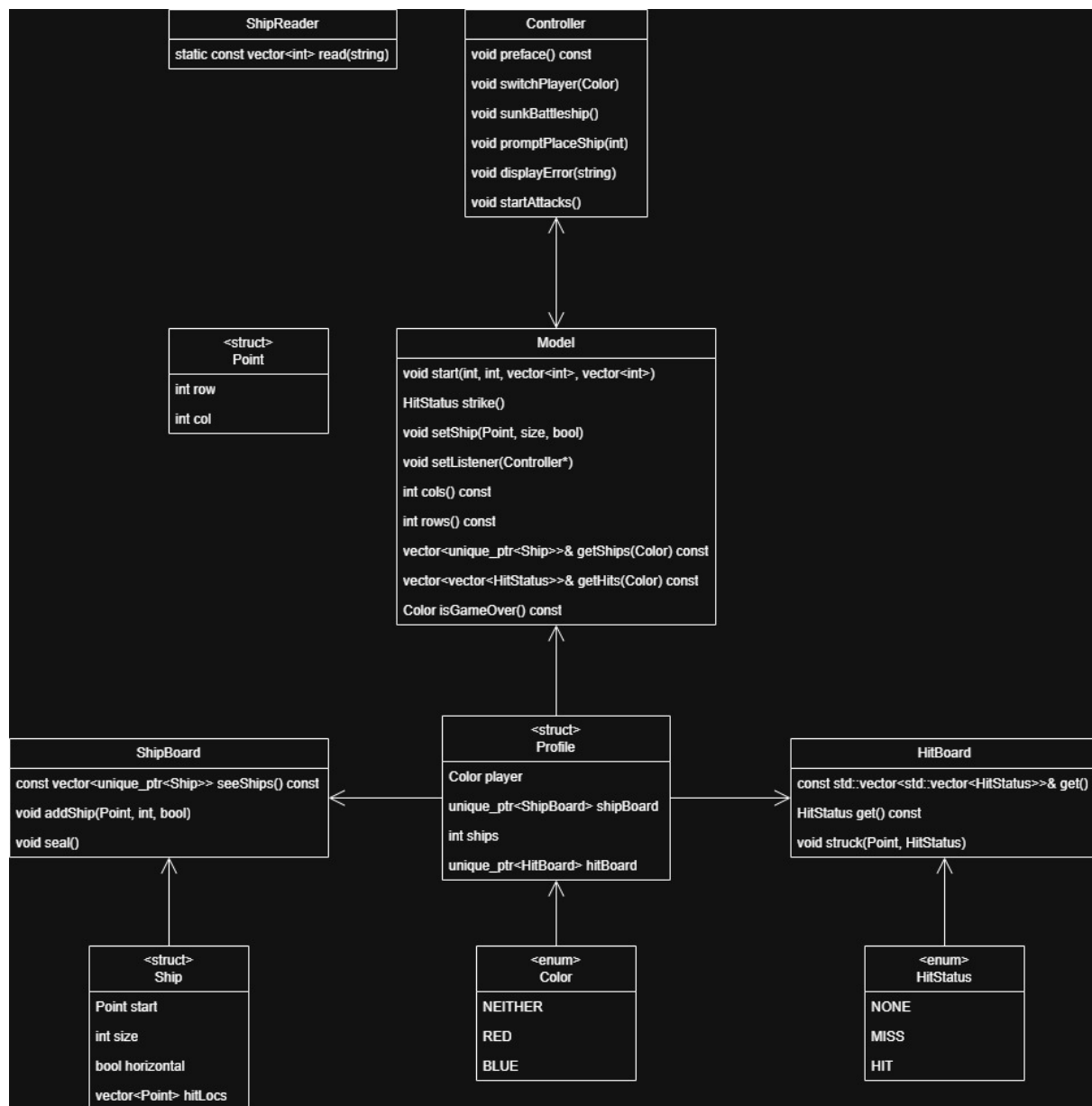
- ✓ **≥ 4 Classes:** Controller, Model, ShipBoard, HitBoard, plus four enums and structs — each with a distinct, single responsibility.
- ✓ **Private Attributes:** All data members are `private`; external access is only through public methods.
- ✓ **Encapsulation:** State maintenance is completely hidden behind user interface.
- ✓ **Rule of 3/5:** Each class defines own destructor; move semantics are leveraged through `unique_ptr`.

4 System Overview — Class Relationships

4.1 High-Level Description

All object interactions in the game adhere to traditional object-oriented design principles and employ a liberal Model-View-Controller architecture. All user interaction interfaces with the Controller, which then manipulates the Model. Due to time constraints, and the fact that the program is terminal-based and synchronous, the View and Controller were merged into one.

4.2 UML Class Diagram



4.3 Detailed Description

The model and controller utilize pub/sub to interact with each other in an encapsulated, decoupled way. The Controller handles all of the user inputs and interactions and relays their requests to the model. The model verifies the data, raises exceptions to the Controller, or otherwise makes the requested changes. In the case of the ship placement phase, the game starts when the Model requests the Controller for placement information from the user. The Controller then makes a callback to the Model with the user information it receives. Inside the Model, game state is primarily located within its two composed Profiles, since each player has their own HitBoard, ShipBoard, and condition for losing. The Model owns the profiles, and the Profiles own the boards.

5 Pseudo Code

</> Main Gameplay Loop

```
1 PROGRAM Battleship
2   INPUT numRows, numCols, redShipsFilepath, blueShipsFilepath
3   INITIALIZE playerProfiles (hitBoard, shipBoard, ships)
4
5   // Phase 1: prompt user placement of all of their ships on their
   //           respective boards
6   FOR redPlayer, bluePlayer:
7     FOR ship IN playerShips:
8       INPUT row, col, orientation
9       VERIFY row, col, orientation
10      addShip(row, col, orientation)
11    END FOR
12    acknowledgeNewPlayer()
13  END FOR
14
15  // Phase 2: alternate attacks until someone wins
16  gameNotOver = true
17  WHILE gameNotOver:
18    acknowledgeNewPlayer()
19    INPUT row, col
20    VERIFY row, col
21    strike(row, col)
22    REPORT strike
23    IF player.ships == 0:
24      gameNotOver = false
25    END IF
26    acknowledgeNewPlayer()
27  END WHILE
28
29  // Phase 3: calculate winner, display, exit
30  PRINT winner
31 END PROGRAM
```

6 C++ Implementation Highlights

</> Model::strike() - Core C++ Implementation

```

1  HitStatus Model::strike(Point pos) {
2      checkGameStarted();
3      if (pos.row < 0 || pos.row >= numRows || pos.col < 0 || pos.col
4          >= numCols) {
5          throw IllegalOperation("Requested strike position is out of
6              bounds");
7      }
8      // cannot strike same location twice
9      if (activeProfile->hitBoard->get(pos) != HitStatus::NONE) {
10         throw IllegalOperation("Already struck this location");
11     }
12     // checks each point on each ship for a match
13     auto& ships = inactiveProfile->shipBoard->seeShips();
14     for (auto& ship : ships) {
15         for (int off = 0; off < ship->size; off++) {
16             int row = ship->start.row
17                 + (ship->horizontal ? 0 : off);
18             int col = ship->start.col
19                 + (ship->horizontal ? off : 0);
20             if (pos.row == row && pos.col == col) {
21                 // on hit: update attacker view,
22                 // reduce ship health, switch players, report hit
23                 activeProfile->hitBoard->
24                     struck(pos, HitStatus::HIT);
25                 ship->hitLocs.push_back(pos);
26                 if (ship->hitLocs.size() == ship->size) {
27                     inactiveProfile->ships--;
28                     controller->sunkBattleship();
29                 }
30                 switchActive();
31                 return HitStatus::HIT;
32             }
33         }
34     }
35     // if nothing found, miss
36     activeProfile->hitBoard->struck(pos, HitStatus::MISS);
37     switchActive();
38     return HitStatus::MISS;
39 }

```

⚙️ Key Design Decisions

- D1. Observer pattern for model and controller:** Frames controller as a listener to model behavior and state change, and as an interface through which players inject information into the model.
- D2. Custom exception `IllegalOperation`:** Allowed for consistent and reliable way to catch relevant user-input related errors for the controller to catch and report to the user in a graphical way.

D3. Player info encapsulation through `Profile` struct: `Model` maintained a `Profile` for each player, containing information about each of their boards and how many ships they had remaining. Pointers to each of these `activeProfile` and `inactiveProfile` allowed for flexible code reuse by adjusting the pointers (instead of using conditional branching).

D4. `Ship` structs: Each player maintained a list `Ship` structs; each `Ship` contained its own "head" position, length and orientation, allowing for consistent deduction of all reached points.

7 Challenges, Solutions & Lessons Learned

7.1 Challenges & Solutions

#	Challenge	Solution
1	<code>Ships</code> occupy multiple cells, but must be encapsulated within a single state.	Abandon 2D grid internal structure and maintain 1D list of <code>Ships</code> that keep track of their own position; ensure valid state at insertion time.
2	Two players must share a screen in a game with secret information.	Introduced a buffer screen that required acknowledgment from both players before proceeding, ensuring neither player accidentally learns more than they should.
3	Enum values cannot be null and shouldn't be subjects of pointers	Included "null-like" falsy values <code>NONE</code> and <code>NEITHER</code> for <code>HitStatus</code> and <code>Color</code> respectively.
4	Constantly passing around object references introduced complex data ownership issues.	Merged several classes and minimized use of raw pointers whenever possible, opting to utilize <code>unique_ptrs</code> with move semantics.

7.2 Lessons Learned

- ★ **Follow your roadmap:** OOP encapsulation allows for easy-to-follow planning and execution from the outset.
- ★ **Documentation is enables efficiency:** consistent and unambiguous documentation allows everyone to develop the project in parallel.
- ★ **Limit your scope:** excessive ambition leads to an over-engineered and excessively complicated project.
- ★ **Data ownership matters:** adhering to ownership conventions and using `unique_ptr`

when applicable saved future memory issues.

- ★ **Graphics takes vision:** designing graphical interfaces requires strict attention to detail and iterative engineering practices.

8 Limitations & Future Work

Current Limitation	Proposed Future Extension
Console-only view.	Integrate Qt for more user-friendly graphical display.
Offline games only.	Integrate gameplay over network so players don't need to share a device.
Lack of AI opponents.	Implement AI players that follow particular heuristics and strategies for an engaging single-player experience.
Lack of interfaces.	Abstract out all behavior into interface classes for improved modularity with rulesets, graphics, and other implementations.
Weak encapsulation.	Redesign <code>Controller.h</code> to exclusively have the responsibilities of a controller, and implement an independent view class.
Fixed two players only.	Redesign state maintenance systems to allow for a flexible number of players per game.

9 GitHub Repository Structure

</> Repository Layout

```

battleship/
|-- src/
|   |-- include/
|   |   |-- Controller.h
|   |   |-- data.h
|   |   |-- HitBoard.h
|   |   |-- Model.h
|   |   |-- ShipBoard.h
|   |   |-- ShipReader.h
|   |-- battleship.cpp      % main program
|   |-- Controller.cpp
|   |-- HitBoard.cpp
|   |-- Model.cpp
|   |-- ShipBoard.cpp
|   |-- ShipReader.cpp
|-- docs/
|   |-- invalid.ships
|   |-- standard.ships
|   |-- tiny.ships
|   |-- TechnicalReport.pdf % this document
|-- src-old/
|   |-- ...      % 20+ classes preserved for posterity
|-- Makefile
'-- README.md

```

A Sample Program Output

</> Single-Player View during Placement Phase

```

+-----+
|      1  2  3  4  5  6  7  8  9  10 |
| A  0  0  0  0  0  0  0  0  0  0 |
| B  0  0  0  0  0  0  0  0  0  0 |
| C  0  0  0  0  0  0  0  0  0  0 |
| D  0  0  0  0  0  0  0  0  0  0 |
| E  0  0  0  0  0  0  0  0  0  0 |
| F  0  0  0  0  0  0  0  0  0  0 |
| G  0  0  0  0  0  0  0  0  0  0 |
| H  0  0  0  0  0  0  0  0  0  0 |
| I  0  0  0  0  0  0  0  0  0  0 |
| J  0  0  0  0  0  0  0  0  0  0 |
+-----+
|      1  2  3  4  5  6  7  8  9  10 |
| A  X  X  0  0  0  0  0  0  0 |
| B  0  0  0  0  0  0  0  0  0  0 |

```

```
| C | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | | |
| D | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| E | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| F | 0 | 0 | 0 | 0 | 0 | X | 0 | 0 | 0 | 0 | 0 | |
| G | 0 | 0 | 0 | 0 | 0 | X | 0 | 0 | 0 | 0 | 0 | |
| H | 0 | 0 | 0 | 0 | 0 | X | 0 | 0 | 0 | 0 | 0 | |
| I | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| J | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
+-----+
Enter location and orientation for a 3-long ship:
```

B Data Maintenance

 data.h - public enums and structs

```
1  #ifndef DATA_H
2  #define DATA_H
3
4  #include <memory>
5  #include <stdexcept>
6
7  struct Point {
8      int row;
9      int col;
10 };
11
12 enum Color {
13     NEITHER,
14     RED,
15     BLUE
16 };
17
18 enum HitStatus {
19     NONE,
20     MISS,
21     HIT
22 };
23
24 struct IllegalOperation : std::runtime_error {
25     using std::runtime_error::runtime_error;
26 };
27
28 #endif
```